

Problem 2: Character-Level Indian Name Generation Using RNN Variants

Vanilla RNN, Bidirectional LSTM, RNN with Attention (PyTorch)

Shivam Madhav Kenche

Roll No.: M25CSA028

March 25, 2026

Abstract

This report describes dataset construction, three recurrent architectures for next-character prediction, quantitative metrics (novelty rate and diversity), and qualitative analysis of generated Indian-style names. Implementations use PyTorch; BLSTM uses prefix-aligned training and prefix decoding; the attention model uses causal prefix pooling and prefix decoding for aligned inference.

Contents

1 Task 0: Dataset	1
2 Task 1: Models and training	1
2.1 Vanilla RNN	2
2.2 Bidirectional LSTM (BLSTM)	2
2.3 RNN with additive attention	2
2.4 Shared hyperparameters	2
3 Task 2: Quantitative evaluation	2
4 Task 3: Qualitative analysis	3
4.1 Realism	3
4.2 Representative samples	3
4.3 Failure modes	3
5 Conclusion	3

1 Task 0: Dataset

Names were collected into `TrainingNames.txt` (one name per line), targeting at least 1000 Indian given names via an LLM-assisted script (`generate_dataset.py`) with multiple regional/demographic prompts, deduplication, and light cleaning.

2 Task 1: Models and training

All models perform character-level language modelling with teacher forcing: input sequence $\langle \text{SOS}, c_1, \dots, c_n \rangle$ and targets $c_1, \dots, c_n, \langle \text{EOS} \rangle$. Vocabulary includes PAD, SOS, EOS and all characters in the training set (vocabulary size ≈ 55 in this run).

Table 1: Dataset summary (see repository for full details).

Property	Value
Total names	1 076
File	<code>TrainingNames.txt</code>
Approx. mean length	~ 7 characters

2.1 Vanilla RNN

Embedding ($d = 64$) $\rightarrow$ `nn.RNN` ($h = 256$, 2 layers, dropout 0.3) $\rightarrow$ linear to vocabulary.
Trainable parameters: 231 671.

2.2 Bidirectional LSTM (BLSTM)

Embedding ($d = 64$) $\rightarrow$ `nn.LSTM` bidirectional ($h = 128$ per direction, 2 layers, dropout 0.3) $\rightarrow$ linear ($2h \rightarrow |V|$). Training uses *causal prefix loss* (see `train.py`): at each position the loss is computed from a forward pass on the prefix only, matching decoding. Generation uses *prefix re-encoding* each step. **Trainable parameters:** 611 575.

2.3 RNN with additive attention

Embedding $\rightarrow$ `nn.RNN` ($h = 256$, 2 layers) $\rightarrow$ at each time step t , additive (Bahdanau-style) attention over hidden states $h_0, \dots, h_t$ only (causal), concatenated with h_t , then linear to vocabulary. Decoding uses the same prefix re-encoding as BLSTM. **Trainable parameters:** 311 799.

2.4 Shared hyperparameters

Adam optimizer, learning rate 10^{-3} , batch size 32, gradient clip norm 1.0, `ReduceLRonPlateau` scheduler, early stopping (patience 15), up to 100 epochs. Training/validation split: 90%/10%.

3 Task 2: Quantitative evaluation

For each model, 200 names were generated with temperature 0.8. Metrics (from `evaluate.py`, stored in `results/evaluation_results.json`):

- **Novelty rate:** percentage of generated names not present in the training set (case-insensitive).
- **Diversity:** (number of unique generated strings) / (total generated).

Table 2: Measured metrics on the evaluation run archived in the repository JSON.

Model	Novelty (%)	Diversity	Avg. len. ($\pm$ std)	Char. coverage
VanillaRNN	92.0	0.975	6.37 ± 1.39	0.9615
BLSTM	90.5	0.985	6.43 ± 1.57	1.0000
AttentionRNN	92.0	0.990	6.86 ± 1.82	1.0000

Note: Re-running `generate_names.py` resamples from the models, so novelty and diversity can shift slightly; re-run `evaluate.py` for a consistent table before final submission.

4 Task 3: Qualitative analysis

4.1 Realism

All three models produce short strings that read like plausible Indian given names (blends of common stems and endings), with occasional misspellings or unlikely consonant clusters typical of small character-level LMs.

4.2 Representative samples

Examples from `generated/` (same evaluation run as Table 2):

VanillaRNN. Suthan, Amutra, Umayal, Anindi, Amisha, Teeta, Amarti, Amaraman, Ramindean, Aljitsan.

BLSTM. Shureth, Naedeeep, Darpan, Sharamti, Sooni, Sumar, Aanya, Prija, Bhavekh, Jetya.

AttentionRNN. Ramanira, Onkav, Darvipree, Rajan, Veera, Anna, Hertendra, Amandik, Sunal, Mohif.

4.3 Failure modes

- Rare early EOS: very short outputs.
- Odd spellings or consonant clusters: limited data and sampling noise.
- High temperature increases randomness and garbage substrings.

5 Conclusion

Vanilla RNN, causal BLSTM with prefix training/decoding, and causal attention RNN with prefix decoding all achieve high diversity and strong novelty on this dataset. BLSTM carries the largest parameter count; Vanilla RNN is the lightest. Attention matches the others once training and decoding are aligned causally.

Declaration

The code artifacts (PyTorch models, training, generation, evaluation, and plots) are included in the submission bundle alongside this report.